Deployment Descriptor — `web.xml`

The **`web.xml`** file is an important configuration file used in **Java Web Applications** and is also called the **Deployment Descriptor**.

Its main purpose is to tell the web server/container **which Servlet should handle a particular URL request**.

1. Introduction

When a client interacts with a Java web application, the client may request different URLs such as:

- `/mylogin`
- `/myregister`
- `/home`

The web application may contain many Servlet classes, such as:

- `Login.java`
- `Register.java`
- `Home.java`

The server needs to know which Servlet should execute for each requested URL.

The **`web.xml`** **file provides this mapping information**.

Definition

Deployment Descriptor (`web.xml`): A configuration file in a Java web application that defines how the web application, including its Servlets, should be configured and mapped.

2. Purpose of `web.xml`

The major purpose of `web.xml` is **Servlet Mapping**.

It creates a connection between:

Client URL → Logical Servlet Name → Actual Servlet Class

For example:

```
/mylogin

   ↓
```

```
mylog

   ↓

in.sp.backend.Login
```

Therefore, when the client requests `/mylogin`, the server knows that it should execute the `Login` Servlet.

Key Points

- `web.xml` contains configuration information for the web application.
- It can define **Servlet mappings**.
- It connects a URL pattern with a Servlet class.
- It acts as a configuration map for the web server.
- A single `web.xml` can contain mappings for multiple Servlets.

3. Location of `web.xml`

The `web.xml` file is normally placed inside the **`WEB-INF`** directory.

A traditional project structure looks like:

```
Web Application
|
├── Web Content / Webapp
|   |
|   ├── WEB-INF
|   |   └── web.xml
|   |
|   ├── index.html
|   └── other files
|
└── Java Resources
    └── Java packages
        └── Servlet classes
```

Important for Exam

`web.xml` is placed inside the `WEB-INF` folder.

4. Basic Structure of `web.xml`

The configuration is written using **XML tags**.

The main/root element is:

```
<web-app>

    <!-- Configuration goes here -->

</web-app>
```

Inside `<web-app>`, we can define Servlet configuration and mappings.

5. `<servlet-mapping>` Tag

The **`<servlet-mapping>`** element specifies **which URL pattern is associated with a particular Servlet name**.

Example:

```
<servlet-mapping>

    <servlet-name>mylog</servlet-name>

    <url-pattern>/mylogin</url-pattern>

</servlet-mapping>
```

It contains two important elements:

5.1 `<servlet-name>`

The `<servlet-name>` gives the Servlet a **logical name**.

Example:

```
<servlet-name>mylog</servlet-name>
```

`mylog` is a name chosen for the Servlet configuration.

It is used to connect the `<servlet-mapping>` section with the corresponding `<servlet>` section.

5.2 `<url-pattern>`

The `<url-pattern>` specifies the URL through which the Servlet can be accessed.

Example:

```
<url-pattern>/mylogin</url-pattern>
```

If the application receives a request for `/mylogin`, the server looks for the Servlet mapped to this URL.

Remember: In this type of mapping, the URL pattern starts with `/`.

6. `<servlet>` Tag

The **`<servlet>`** element defines the actual Servlet class associated with the logical Servlet name.

Example:

```
<servlet>
    <servlet-name>mylog</servlet-name>
    <servlet-class>in.sp.backend.Login</servlet-class>
</servlet>
```

It contains two important elements.

6.1 `<servlet-name>`

This must match the Servlet name used in `<servlet-mapping>`.

For example:

```
<servlet-mapping>
    <servlet-name>mylog</servlet-name>
    ...
</servlet-mapping>
```

and:

```
<servlet>
    <servlet-name>mylog</servlet-name>
    ...
</servlet>
```

Both use:

```
mylog
```

6.2 `<servlet-class>`

The `<servlet-class>` specifies the **actual Java Servlet class**.

Example:

```
<servlet-class>in.sp.backend.Login</servlet-class>
```

Here:

- `in.sp.backend` → package name
- `Login` → Servlet class name

7. Complete Servlet Mapping

A complete mapping connects three things:

```
URL Pattern

    ↓

Servlet Name

    ↓

Servlet Class
```

For example:

```
/mylogin

    ↓

mylog

    ↓

in.sp.backend.Login
```

The corresponding XML is:

```
<web-app>


    <servlet-mapping>

        <servlet-name>mylog</servlet-name>

        <url-pattern>/mylogin</url-pattern>

    </servlet-mapping>
```

```
    <servlet>
        <servlet-name>mylog</servlet-name>
        <servlet-class>in.sp.backend.Login</servlet-class>
    </servlet>


</web-app>
```

How it works

Suppose the user requests:

`/mylogin`

The server performs the following logical process:

1. It finds `/mylogin` in `<url-pattern>`.
2. It sees that the associated Servlet name is `mylog`.
3. It searches for the `<servlet>` configuration having the same name, `mylog`.
4. It finds `in.sp.backend.Login`.
5. The server uses the **`Login` Servlet** to handle the request.

8. Mapping Multiple Servlets

A web application can contain multiple Servlets.

For example:

- `Login.java`
- `Register.java`

Each Servlet can have its own URL mapping.

Example:

```
<web-app>


    <servlet-mapping>
        <servlet-name>mylog</servlet-name>
        <url-pattern>/mylogin</url-pattern>
```

```
    </servlet-mapping>


    <servlet>
        <servlet-name>mylog</servlet-name>
        <servlet-class>in.sp.backend.Login</servlet-class>
    </servlet>




    <servlet-mapping>
        <servlet-name>myreg</servlet-name>
        <url-pattern>/myregister</url-pattern>
    </servlet-mapping>


    <servlet>
        <servlet-name>myreg</servlet-name>
        <servlet-class>in.sp.backend.Register</servlet-class>
    </servlet>


</web-app>
```

The mapping becomes:

URL	Servlet Name	Servlet Class
/mylogin	mylog	in.sp.backend.Login
/myregister	myreg	in.sp.backend.Register

Important

For every Servlet, the **`servlet-name` must match between its `<servlet>` and `<servlet-mapping>` elements**.

9. `web.xml` vs Annotations

Traditionally, `web.xml` was commonly used to configure Servlets.

Modern Servlet applications can also use **annotations**, such as `@WebServlet`, directly in the Java Servlet class.

For example:

```
@WebServlet("/mylogin")

public class Login extends HttpServlet {

    // Servlet code

}
```

This can define the URL mapping without manually putting that mapping into `web.xml`.

Comparison

Feature	`web.xml`	`@WebServlet`
Type	XML configuration	Java annotation
Location	`WEB-INF/web.xml`	Java Servlet class
Servlet mapping	Defined in XML	Defined in Java code
Common use	Traditional/legacy projects and explicit centralized configuration	Modern Servlet applications

Important for Exam

`web.xml` is not strictly required in every modern Servlet application. Servlet annotations can be used for many configurations instead.

However, `web.xml` is still important for understanding **older Java web applications and projects that use deployment descriptors**.

10. Important Terms

Deployment Descriptor

A configuration file that describes how a Java web application should be deployed and configured.

Servlet Mapping

The process of associating a URL pattern with a Servlet.

URL Pattern

The URL path used to access a particular Servlet.

Example:

```
<url-pattern>/mylogin</url-pattern>
```

Servlet Name

A logical name used to connect the Servlet mapping with its Servlet configuration.

Servlet Class

The actual Java class that implements the Servlet.

Example:

```
<servlet-class>in.sp.backend.Login</servlet-class>
```

11. Important Exam Points

Remember:

- **`web.xml` = Deployment Descriptor.**
- It is traditionally located inside **`WEB-INF`**.
- It can be used for **Servlet configuration and mapping**.
- `<servlet-mapping>` associates a URL pattern with a logical Servlet name.
- `<servlet>` associates the logical Servlet name with the actual Servlet class.
- The `<servlet-name>` in both sections must match.
- The `<url-pattern>` normally begins with `/`.
- Multiple Servlets can be configured in the same `web.xml`.
- Modern Servlet applications can often use annotations such as **`@WebServlet`** instead.

12. Commonly Confused

`<servlet>` vs `<servlet-mapping>`

`<servlet>`	`<servlet-mapping>`
Defines the Servlet	Defines how the Servlet is accessed
Identifies the actual Servlet class	Identifies the URL pattern
Uses `<servlet-class>`	Uses `<url-pattern>`
Example: `Login` class	Example: `/mylogin`

A simple way to remember:

`<servlet>` → Which Java class?
`<servlet-mapping>` → Which URL?

13. Overall Working

The complete concept can be remembered as:

```
                CLIENT
                  |
                  | requests /mylogin
                  ↓
            URL PATTERN
            /mylogin
                  |
                  ↓
          SERVLET NAME
             mylog
                  |
                  ↓
          SERVLET CLASS
    in.sp.backend.Login
                  |
                  ↓
       LOGIN SERVLET
```

So, `web.xml` essentially provides the information needed to connect the **client's requested URL** to the **correct Servlet class**.

14. Key Points

- **`web.xml`** is called the **Deployment Descriptor**.
- It is traditionally stored in **`WEB-INF`**.

- Its important role is **Servlet configuration and URL mapping**.
- `<servlet-mapping>` connects a **URL pattern** to a **Servlet name**.
- `<servlet>` connects the **Servlet name** to the **actual Servlet class**.
- The same `<servlet-name>` must be used in both sections.
- Multiple Servlets can be configured in one `web.xml`.
- Modern applications can use **annotations such as `@WebServlet`** instead of XML for many Servlet mappings.

Summary

The **`web.xml` Deployment Descriptor** is a configuration file traditionally used in Java web applications. Its most important function in Servlet-based applications is to map a client's **URL pattern** to the appropriate **Servlet class**. The `<servlet-mapping>` element identifies the URL and logical Servlet name, while the `<servlet>` element connects that logical name to the actual Java Servlet class. Although annotations such as `@WebServlet` are widely used in modern applications, understanding `web.xml` remains important for exams and for working with traditional Java web projects.

Video Title: #5 Deployment Descriptor || web.xml file in Servlet Eclipse || Servlet JSP Tutorials
Video link: https://youtu.be/-xWADaceiwI?si=SaVplpcGb8lqryNl